

1) Introduction

This assignment focuses on the implementation and experimental analysis of divide and conquer algorithms and asymptotic complexity. Three main algorithms were implemented in Java: MergeSort, QuickSort and QuickSelect. In addition, InsertionSort was used as a cutoff optimization for small subarrays in MergeSort.

The main objectives of the assignment were to implement the algorithms correctly, analyze their theoretical time complexity, measure their practical performance, and compare the experimental results with the expected asymptotic behavior.

The implementation also includes a Metrics class that records the number of comparisons and the maximum recursion depth. The algorithms were tested using JUnit 5 on random arrays and several edge cases. A benchmark was performed using input sizes of 1,000, 10,000, 100,000, and 1,000,000 elements.

Complexity Table:

Algorithm	Best Case	Average Case	Worst Case
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
QuickSort	$O(n \log n)$	$O(n \log n)$ expected	$O(n^2)$
QuickSelect	$O(n)$	$O(n)$ expected	$O(n^2)$
InsertionSort	$O(n)$	$O(n^2)$	$O(n^2)$

MergeSort has $O(n \log n)$ complexity in the best, average, and worst cases because the array is always divided into two parts and the merging operation requires linear time.

QuickSort has an expected complexity of $O(n \log n)$ when the partitions are reasonably balanced. However, its worst-case complexity is $O(n^2)$ when the partitions become highly unbalanced. Random pivot selection reduces the probability of consistently bad partitions, but it does not remove the theoretical worst case.

QuickSelect is based on the same partitioning idea as QuickSort, but it continues only with the partition that contains the required element. Therefore, its expected complexity is $O(n)$, while its worst-case complexity is $O(n^2)$.

InsertionSort has $O(n)$ best-case complexity when the array is already sorted. Its average and worst-case complexities are $O(n^2)$.

3) Recurrences

For MergeSort:

$$T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$$

For balanced QuickSort:

$$T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$$

For balanced QuickSelect:

$$T(n) = T(n/2) + O(n) \rightarrow O(n)$$

QuickSort and QuickSelect can have $O(n^2)$ worst-case complexity when partitions are highly unbalanced.

4) Benchmark Setup

The algorithms were tested with input sizes of 1,000, 10,000, 100,000, and 1,000,000 elements.

Three input types were used:

- Random
- Sorted
- Duplicate values from 0 to 9

Each test was repeated five times, and the median result was recorded. The benchmark measured execution time, comparisons, and maximum recursion depth.

5) Experimental Results

1-Running time

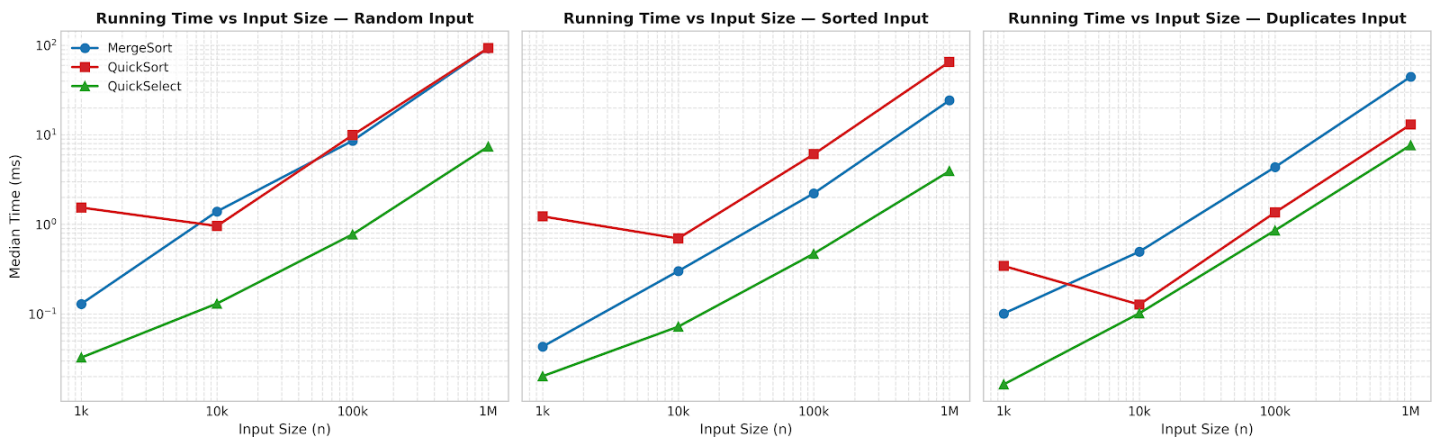


Figure 1. Running Time vs Input Size

The running time increases as the input size grows. MergeSort and QuickSort show behavior consistent with $n \log n$ growth, while QuickSelect shows approximately linear growth for large inputs. QuickSort performs particularly well on duplicate-heavy input because of three-way partitioning.

2-Maximum Recursion Depth

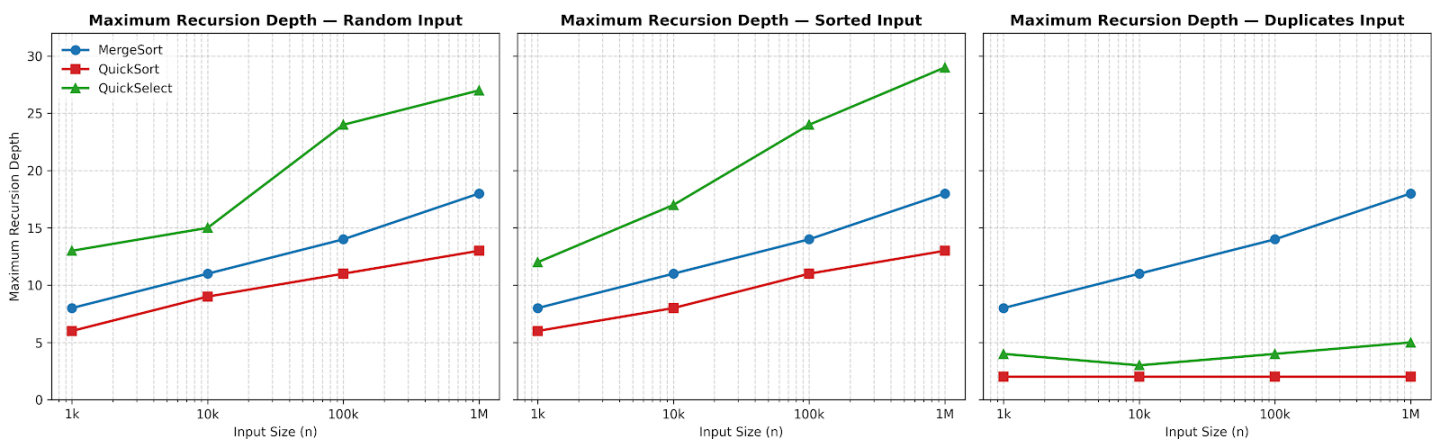


Figure 2. Maximum Recursion Depth

MergeSort has predictable logarithmic recursion depth. QuickSort maintains a relatively small recursion depth because it recursively processes the smaller partition and processes the larger partition iteratively. The maximum observed QuickSort depth was 13 for 1,000,000 elements.

3-Asymptotic Ratio

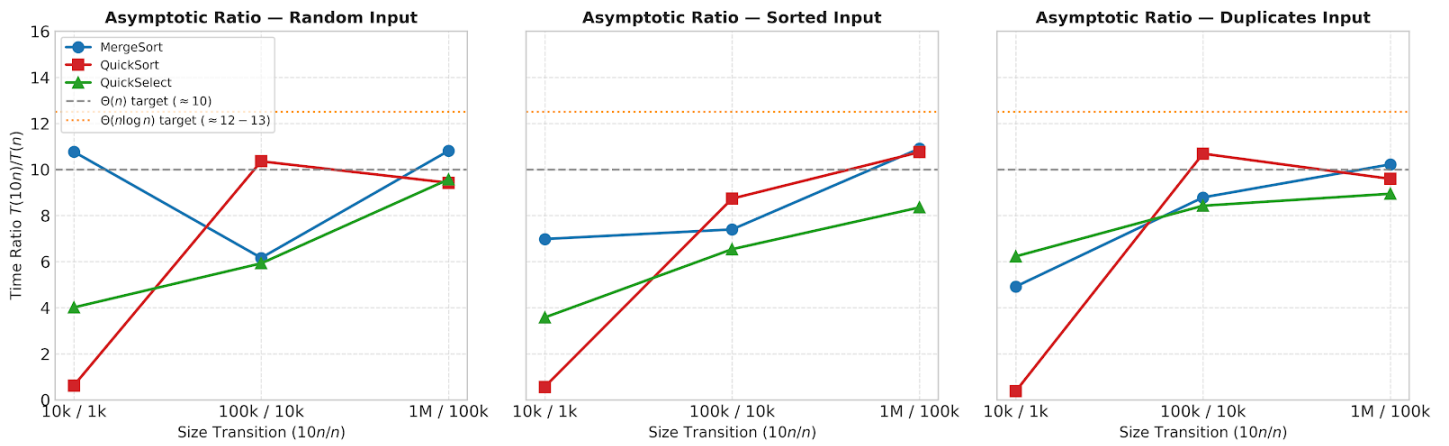


Figure 3. Asymptotic Ratio $T(10n)/T(n)$

QuickSelect ratios for large inputs are close to 10, which is consistent with linear-time behavior. MergeSort and QuickSort show ratios generally compatible with $n \log n$ growth. Small-input ratios are affected by JVM overhead and measurement noise.

5. Discussion

The experimental results are generally consistent with the theoretical complexity of the algorithms. Input structure affects practical execution time, especially for QuickSort. Three-way partitioning significantly improves performance on arrays with many duplicate values.

The results also demonstrate that implementation details can affect practical performance and recursion depth without changing the theoretical worst-case complexity.

6. Conclusion

The assignment demonstrated the theoretical and practical behavior of MergeSort, QuickSort, and QuickSelect. The benchmark results generally matched the expected asymptotic complexities. The experiments also showed the importance of input structure and implementation techniques when evaluating algorithm performance.